

Testkonzept für das Open-Source-Projekt „lazygit„

Belegarbeit im Studiengang Informatik (B.Sc.)

**Grundlagen des Softwaretestens
Wintersemester 2025/26**

Dozent: Prof. Dr. Matthias Längrich

Hochschule Zittau/Görlitz
Fakultät Elektrotechnik und Informatik

**Verfasser: Konrad Miosga
Matrikelnummer: 1056710
E-Mail-Adresse: konrad.miosga@stud.hszg.de**

Datum der Abgabe: 15.01.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Projektkontext	1
2. Theoretische Grundlagen	1
3. Analyse der vorhandenen Teststrategie	2
3.1. Testinfrastruktur	2
3.2. Continuous Integration	4
3.3. Code-Coverage-Analyse	4
4. Testumgebung und Werkzeuge	5
4.1. Verwendete Tools	5
4.2. Testumgebungen	5
4.3. Docker-Container für isolierte Tests	5
5. Entwurf eigener Testfälle	6
5.1. Identifikation von Testlücken	6
5.2. Testfalldesign und Methodik	6
5.3. Implementierung der Testfälle	7
5.4. Testergebnisse	8
6. Testauswertung und Metriken	9
6.1. Coverage-Verbesserung	9
6.2. Testmetriken	9
6.3. CI/CD-Integration	9
7. Empfehlungen und Fazit	9
7.1. Bewertung der Teststrategie	9
7.2. Handlungsempfehlungen	10
7.3. Fazit	10
Quellen	10

1. Einleitung

1.1. Projektkontext

In dieser Belegarbeit wird das Testkonzept des Open-Source-Projekts **lazygit** analysiert, bewertet und durch eigene Testfälle ergänzt. Lazygit ist eine in Go entwickelte Terminal-UI für Git-Kommandos, die komplexe Git-Operationen durch eine intuitive, tastaturgesteuerte Benutzeroberfläche vereinfacht. Das Projekt ist Open Source, community-getrieben und wird aktiv genutzt.

2. Theoretische Grundlagen

Die statische Analyse umfasst Prüftechniken, bei denen Software ohne Programmausführung untersucht wird. Sie kann keine vollständigen Aussagen über Korrektheit treffen, wird aber werkzeugunterstützt für Software-Messungen, Stilanalysen und Datenflussanomalieanalyse eingesetzt. [1, S. 43-44] Diese Arbeit fokussiert jedoch auf dynamische Testtechniken.

Der dynamische Test führt die Software mit konkreten Eingabedaten aus. Als Stichprobenverfahren kann er Fehler aufzeigen, aber keine Korrektheit beweisen. Ziel ist die Auswahl repräsentativer, fehlersensitiver und wirtschaftlicher Testfälle für aussagekräftige Ergebnisse. [1, S. 39-43] Bei lazygit werden Unit-Tests, Integrationstests und ein spezialisiertes End-to-End Test-Framework eingesetzt.

Black-Box-Tests konstruieren Testfälle ausschließlich aus Anforderungen und Spezifikationen ohne Kenntnis der internen Struktur. White-Box-Tests leiten Testfälle aus der Programmstruktur ab, um vollständige Code-Abdeckung zu erreichen. [2, S. 173-174] In der Praxis werden beide Ansätze kombiniert eingesetzt.

Unit-Tests prüfen kleinste testbare Einheiten isoliert. Die zu testende Einheit wird aus ihrem Kontext gelöst und abhängige Komponenten durch Mocks oder Stubs ersetzt. Dies ermöglicht frühzeitige Fehlererkennung, deckt aber keine Integrationsfehler auf. [1, S. 371-372]

Table-Driven Tests fassen mehrere Testfälle in Datentabellen zusammen. Jede Zeile definiert Eingaben und erwartete Ausgaben. Ein einziger Testcode iteriert über alle Einträge, was redundante Tests vermeidet und die Wartbarkeit erhöht. [3]

Integrationstests prüfen das Zusammenwirken bereits getesteter Module. Sie decken Schnittstellenfehler auf, die bei isolierten Unit-Tests nicht sichtbar werden. Die Integration erfolgt nach festgelegten Strategien wie Bottom-up oder Top-down. [1, S. 372-376]

3. Analyse der vorhandenen Teststrategie

Lazygit setzt verschiedene Teststrategien ein, die ein umfassendes Sicherheitsnetz für die Entwicklung bilden. Die Teststrategie folgt dem Pyramidenmodell mit einer breiten Basis von Unit-Tests, einer mittleren Schicht von Integrationstests und punktuellen End-to-End-Tests.

3.1. Testinfrastruktur

Das Projekt nutzt das Standard-Go-Testing-Framework als Grundlage. Unit-Tests befinden sich direkt neben dem Produktionscode mit der Namenskonvention `*_test.go`. Diese Ko-Lokation erleichtert die Wartung und stellt sicher, dass Tests bei Änderungen am Code nicht vergessen werden.

Ein Beispiel für einen klassischen Unit-Test findet sich in `pkg/commands/git_commands/branch_test.go`. Der Test verwendet den `FakeRunner`, ein Mock-Objekt, das Git-Befehle simuliert:

```
func TestBranchNewBranch(t *testing.T) {
    runner := oscommands.NewFakeRunner(t).
        ExpectGitArgs([]string{"checkout", "-b", "test", "refs/heads/master"}, "", nil)
    instance := buildBranchCommands(commonDeps{runner: runner})

    assert.NoError(t, instance.New("test", "refs/heads/master"))
    runner.CheckForMissingCalls()
}
```

Dieser Test validiert, dass beim Erstellen eines neuen Branches der korrekte Git-Befehl konstruiert wird. Der `FakeRunner` erwartet spezifische Argumente und liefert vordefinierte Antworten, ohne tatsächlich Git auszuführen. Dies ermöglicht schnelle, deterministische Tests ohne Seiteneffekte.

Table-Driven Tests

Table-Driven Tests werden in lazygit konsequent eingesetzt, insbesondere in den Command-Tests. Dabei wird eine Slice von Test-Szenarien definiert, die jeweils Eingabewerte und erwartete Ausgaben enthalten. Ein gutes Beispiel findet sich in `tag_test.go`, wo verschiedene

Szenarien für die Tag-Erstellung getestet werden. Die Tests definieren eine Struktur mit Testname, Eingabeparametern wie `tagName`, `ref` und `force`, sowie den erwarteten Git-Command-Argumenten. Anschließend wird über alle Szenarien iteriert und für jedes ein eigenständiger Sub-Test ausgeführt. Diese Vorgehensweise ermöglicht es, viele Varianten derselben Funktionalität mit minimalem Code-Overhead abzudecken und neue Testfälle durch einfaches Hinzufügen weiterer Tabelleneinträge zu ergänzen. Die Table-Driven Tests in lazygit folgen den Go-Best-Practices und nutzen `testing.T.Run()` zur Erstellung benannter Sub-Tests, was eine bessere Fehlerdiagnose und selektive Testausführung ermöglicht. [3]

Integration Tests

Die Integrationstests von lazygit testen das Zusammenspiel mehrerer Komponenten und sind in einem eigenen Test-Framework implementiert. Sie verwenden echte Git-Repositories, die in isolierten Sandbox-Umgebungen erstellt werden, um realistische Szenarien zu simulieren. Diese Tests prüfen beispielsweise, ob Commits, Branches und Tags korrekt in der UI angezeigt werden, ob interaktive Rebases funktionieren und ob Git-Operationen die erwarteten Ergebnisse liefern. Die Integrationstests laufen gegen verschiedene Git-Versionen (2.32.0 bis latest) und werden in der CI-Pipeline parallel ausgeführt, um Kompatibilität über verschiedene Git-Versionen hinweg sicherzustellen. Anders als Unit-Tests werden hier keine Mocks verwendet, sondern tatsächliche Git-Befehle in einer kontrollierten Umgebung ausgeführt.

Typ		Blackbox	Whitebox	Hybrid	Test-
Unit	Tests	✗	✓	-	
Command	Tests	✗	✓	-	
Parser	Tests	✗	✓	-	
Integration	Tests	✓	✗	-	



3.2. Continuous Integration

Lazygit nutzt GitHub Actions für umfassende automatisierte Tests bei jedem Push und Pull Request. Die CI-Pipeline umfasst mehrere Jobs, die parallel ausgeführt werden und verschiedene Aspekte der Software testen. Der Unit-Test-Job läuft auf Ubuntu und Windows, wobei Code Coverage-Daten gesammelt werden. Ein separater Integration-Test-Job testet gegen verschiedene Git-Versionen, angefangen bei der ältesten unterstützten Version 2.32.0 bis hin zur neuesten Version. Dies gewährleistet Kompatibilität über einen breiten Bereich von Git-Installationen. Die Workflow-Datei `ci.yml` definiert die Go-Version 1.25 als Standard und nutzt Vendor-Mode für reproduzierbare Builds. Zusätzlich gibt es spezielle Jobs für Code-Linting mit `golangci-lint` und Rechtschreibprüfung mit `codespell`. Die Matrix-Builds ermöglichen es, potenzielle plattformspezifische Probleme frühzeitig zu erkennen, bevor Code in den Master-Branch gemergt wird.

3.3. Code-Coverage-Analyse

Die Code-Coverage wird in der CI-Pipeline automatisch erfasst und als Artefakt gespeichert. Unit-Tests werden mit Coverage-Tracking ausgeführt, wobei die Daten in einem temporären Verzeichnis gesammelt und anschließend hochgeladen werden. Lazygit nutzt das Coverage-Tool von Go, das in der Standard-Testbibliothek integriert ist. Die Tests werden mit dem `-short` Flag ausgeführt, um Unit-Tests von Integrationstests zu trennen, was eine differenzierte Analyse der Testabdeckung ermöglicht. Das Projekt legt besonderen Wert auf die Abdeckung kritischer Komponenten wie Command-Builder, Git-Operations und Controller-Logik. Weniger kritische UI-Rendering-Komponenten haben naturgemäß eine geringere Coverage, da sie schwerer automatisiert testbar sind und häufig manuell validiert werden müssen. Die Coverage-Artefakte werden pro Betriebssystem und Build-Lauf getrennt gespeichert, was eine detaillierte Analyse plattformspezifischer Testabdeckung ermöglicht.

4. Testumgebung und Werkzeuge

4.1. Verwendete Tools

Lazygit nutzt das Standard-Go-Testing-Framework mit dem `testing`-Paket und `go test`-Runner. Ergänzend kommt Testify zum Einsatz, eine Assertion-Bibliothek mit Funktionen wie `assert.Equal()` und `require.NoError()`. Während `assert` bei Fehlschlag den Test weiterlaufen lässt, bricht `require` sofort ab. Für Test-Setup wird die Go-Git-Bibliothek verwendet, um Git-Repositories programmatisch zu manipulieren. Die PTY-Bibliothek (Pseudo-Terminal) ermöglicht die Simulation von Terminal-Interaktionen für End-to-End-Tests der Terminal-UI.

4.2. Testumgebungen

Die Test-Matrix deckt verschiedene Konfigurationen ab:

Komponente	Variante
Betriebssysteme	Ubuntu, Windows
Go-Versionen	1.25
Git-Versionen	2.32.0, 2.38.2, 2.44.0, latest
Terminaltypen	xterm-256color, dumb

Die Git-Versions-Matrix ist besonders wichtig: Version 2.32.0 ist die älteste unterstützte, 2.38.2 die erste mit `rebase.updateRefs`-Support, und latest testet aktuelle Features. Terminal-Typen werden getestet, da verschiedene Terminals unterschiedliche ANSI-Escape-Sequenzen unterstützen.

4.3. Docker-Container für isolierte Tests

Das Projekt bietet Docker-Unterstützung für lokale Entwicklung. Der Container basiert auf Alpine-Linux mit Go und Git und ermöglicht Tests in reproduzierbarer Umgebung. Dev Container-Support erlaubt Entwicklung direkt im Container ohne lokale Umgebungsänderungen. Die Konfiguration im `.devcontainer`-Verzeichnis enthält alle notwendigen Tools und IDE-Extensions.

5. Entwurf eigener Testfälle

5.1. Identifikation von Testlücken

Die Analyse der Testabdeckung erfolgte systematisch durch mehrere Methoden. Zunächst wurden Code-Coverage-Metriken aus der CI-Pipeline ausgewertet, um quantitative Daten über getestete und ungetestete Code-Bereiche zu erhalten. Manuelle Code-Reviews kritischer Komponenten ergänzten diese automatisierten Analysen und identifizierten Bereiche, in denen Tests trotz Coverage fehlen könnten, etwa bei Error-Handling-Pfaden. Die Analyse der Git-Commit-Historie half dabei, kürzlich hinzugefügte oder geänderte Features zu identifizieren, die möglicherweise noch nicht vollständig getestet waren.

Bei der Analyse fielen zwei signifikante Lücken auf. Die Tag-Kommandos in `pkg/commands/git_commands/tag.go` enthielten Funktionen zum Erstellen, Löschen und Inspizieren von Git-Tags, jedoch fehlte `tag_test.go` vollständig. Dies stellte eine signifikante Testlücke dar, da Tag-Operationen zu den Kernfunktionen von Git gehören und in `lazygit` häufig genutzt werden. Ebenso fehlten Tests für die `StringStack`-Datenstruktur in `pkg/utils/string_stack.go`, eine grundlegende Utility-Klasse für Stack-basierte Operationen.

Die Priorisierung erfolgte nach dem Risikoprinzip. Tag-Operationen wurden zuerst adressiert, da sie sowohl häufig genutzt werden als auch bei Fehlfunktion signifikante Probleme verursachen können. Ein fehlerhaft erstellter oder gelöschter Tag kann zu Verwirrung in der Versionsverwaltung führen und ist oft schwer rückgängig zu machen.

5.2. Testfalldesign und Methodik

Für die Tag-Kommandos wurde ein table-driven Ansatz gewählt, der sich bereits in anderen Teilen des Projekts bewährt hatte. Diese Methodik ermöglicht es, viele Testszenarien kompakt und wartbar zu definieren. Jeder Testfall wird als Struktur beschrieben, die Eingabeparameter und erwartete Ausgaben kombiniert.

Die Tests für `CreateLightweightObj` decken vier zentrale Szenarien ab. Erstens das Erstellen eines einfachen Tags auf HEAD ohne spezielle Flags, was den häufigsten Anwendungsfall darstellt. Zweitens das Erstellen eines Tags auf einem spezifischen Commit, identifiziert durch

SHA oder Ref. Drittens die Verwendung des Force-Flags zum Überschreiben existierender Tags, was in der Praxis oft nötig ist, wenn ein Tag versehentlich auf den falschen Commit gesetzt wurde. Viertens die Kombination aus spezifischem Commit und Force-Flag, was alle Parameter-Kombinationen abdeckt. Diese Szenarien wurden bewusst gewählt, um die bedingte Logik in der Implementierung vollständig abzudecken, insbesondere die ArgIf-Konstrukte, die Parameter nur unter bestimmten Bedingungen zum Git-Befehl hinzufügen.

Für annotierte Tags wurde ein analoger Ansatz verfolgt, jedoch mit dem zusätzlichen Parameter `msg` für die Tag-Message. Annotierte Tags enthalten zusätzliche Metadaten wie Autor, Datum und Message, die in Git's Objektdatenbank gespeichert werden. Die Tests stellen sicher, dass diese zusätzlichen Informationen korrekt an Git übergeben werden.

Der Test für `IsTagAnnotated` nutzt einen Mock-basierten Ansatz mit dem `FakeRunner`. Hier werden verschiedene Git-Ausgaben simuliert, einschließlich Whitespace-Varianten. Dies testet die Robustheit des Parsing-Codes gegenüber unterschiedlichen Git-Versionen und Ausgabeformaten. Diese Tests validieren nicht nur die korrekte Ausführung des Git-Befehls, sondern auch die korrekte Interpretation der Ausgabe.

Für die `StringStack`-Tests wurde ein zustandsbasierter Testansatz gewählt. Die Tests validieren nicht nur einzelne Operationen, sondern auch Sequenzen von Push- und Pop-Operationen, um sicherzustellen, dass der interne Zustand konsistent bleibt. Besondere Aufmerksamkeit wurde Edge Cases wie dem Poppen von einem leeren Stack gewidmet, da solche Grenzfälle oft Quelle von Bugs sind.

5.3. Implementierung der Testfälle

Die Implementierung erfolgte in mehreren Schritten und orientierte sich an den Konventionen des Projekts. Zunächst wurde die Datei `pkg/commands/git_commands/tag_test.go` neu erstellt. Die notwendigen Imports umfassten das Standard-Testing-Package, die Testify-Assertion-Bibliothek und interne Packages für OS-Commands und Git-Commands.

Die `TestTagCommands_CreateLightweightObj`-Funktion definiert zunächst eine Struktur für Test-szenarien mit Feldern für `testName`, `tagName`, `ref`, `force` und `expectedCmdArgs`. Anschließend wird eine Slice von Szenarien erstellt, die alle relevanten Kombinationen abdeckt. In der

Testschleife wird für jedes Szenario ein neuer FakeRunner erstellt, der Git-Befehle simuliert ohne sie tatsächlich auszuführen. Dies ermöglicht schnelle, deterministische Tests ohne Abhängigkeiten zur lokalen Git-Installation.

Die eigentliche Testlogik ist kompakt: Sie ruft die zu testende Funktion auf und vergleicht die resultierenden Command-Argumente mit den erwarteten Werten mittels `assert.Equal()`. Diese Assertion-Methode bietet detaillierte Fehlermeldungen, wenn Tests fehlschlagen. Bei einem Mismatch zeigt sie sowohl erwartete als auch tatsächliche Werte an, was das Debugging erheblich erleichtert.

Für `LocalDelete` wurde ein klassischer Mock-basierter Test implementiert, der die `ExpectGitArgs`-Methode des FakeRunners nutzt. Diese Methode definiert exakt, welche Git-Argumente erwartet werden und was als Ausgabe zurückgegeben werden soll. Nach dem Funktionsaufruf verifiziert `CheckForMissingCalls()`, dass alle erwarteten Git-Befehle tatsächlich aufgerufen wurden. Dies stellt sicher, dass keine Befehle übersprungen werden und die Testerwartungen vollständig erfüllt sind.

Die `StringStack`-Tests in `pkg/utils/string_stack_test.go` folgen einem klassischeren Unit-Test-Muster ohne Table-Driven-Ansatz, da hier primär Zustandsübergänge getestet werden. Jeder Test fokussiert auf einen spezifischen Aspekt der Stack-Funktionalität und verwendet aussagekräftige Testnamen wie `TestStringStack_PopEmptyStack` oder `TestStringStack_MultipleOperations`.

Ergänzend zu den Tests wurde eine ausführliche Dokumentation in `TEST_ERKLAERUNG.md` erstellt, die Schritt für Schritt erklärt, wie ein table-driven Test funktioniert. Diese Dokumentation richtet sich an Entwickler, die mit Go-Testing noch nicht vertraut sind, und dient als didaktisches Material. Sie enthält Code-Beispiele, visuelle ASCII-Diagramme und detaillierte Erklärungen jedes Testschritts. Diese Dokumentation trägt zur Wissenserhaltung im Projekt bei und erleichtert neuen Mitwirkenden den Einstieg ins Testing.

5.4. Testergebnisse

Alle 18 implementierten Tests bestanden erfolgreich. Die Tag-Tests mit 13 Sub-Tests deckten `lightweight` und `annotated` Tags ab. Ausführung erfolgte mit `go test ./pkg/`

`commands/git_commands -run TestTag -v`. Alle Szenarien für `CreateLightweightObj` und `CreateAnnotatedObj` funktionierten korrekt. `LocalDelete` validierte den korrekten Git-Befehl, `IsTagAnnotated` bestätigte robustes Parsing. Die fünf `StringStack`-Tests validierten LIFO-Semantik, Edge Cases und Operationssequenzen. Testausführung dauerte unter 10ms ohne externe Abhängigkeiten.

6. Testauswertung und Metriken

6.1. Coverage-Verbesserung

Die neuen Tests erhöhten Coverage für `tag.go` von 0% auf 75-80%, da Hauptfunktionen abgedeckt sind. Remote-Operationen wie `Push` und `Delete` wurden nicht getestet. `string_stack.go` erreichte 100% Coverage. Im Gesamtprojekt ist der Beitrag moderat, aber jeder Test reduziert Regressionsrisiken.

6.2. Testmetriken

Lazygit verfügt über 80+ Test-Dateien im `pkg`-Verzeichnis. Unit-Tests laufen in unter einer Minute. Die Test-Code-Ratio ist ausgewogen, kritische Packages haben umfangreichere Tests als Produktionscode. Table-driven Tests enthalten typischerweise eine Assertion pro Szenario.

6.3. CI/CD-Integration

Die Tests integrierten sich nahtlos in die CI-Pipeline durch Standard-Go-Pattern. Sie laufen auf allen Plattformen ohne CI-Änderungen. GitHub Actions liefert detaillierte Logs mit Stack-Traces. Benannte Sub-Tests ermöglichen schnelle Fehleridentifikation.

7. Empfehlungen und Fazit

7.1. Bewertung der Teststrategie

Lazygit verfügt über eine ausgereifte Teststrategie mit Unit-Tests, Integrationstests und umfassender CI/CD-Integration. Stärken sind die konsequente Verwendung von Mock-Objekten für schnelle, deterministische Tests, table-driven Tests für hohe Wartbarkeit und Matrix-Builds für Plattformkompatibilität. Der `FakeRunner` eliminiert Git-Abhängigkeiten und beschleunigt Tests erheblich.

Schwächen zeigen sich in variierender Coverage zwischen Komponenten. UI-Code ist weniger abgedeckt als Business-Logik, was teilweise durch schwierige UI-Testbarkeit bedingt ist. Ältere Komponenten haben minimale Tests, was Refactoring-Risiken birgt. Test-Dokumentation ist begrenzt, was Einstiegshürden für neue Mitwirkende schafft.

7.2. Handlungsempfehlungen

Folgende Verbesserungen sind empfehlenswert: Systematische Coverage-basierte Identifikation und Schließung von Testlücken, Fokus auf kritische und häufig genutzte Funktionen. Erweiterung der Test-Dokumentation mit zentralem Testing-Guide nach Vorbild von `TEST_ERKLAERUNG.md`. Integration von Coverage-Tools wie Codecov in CI-Pipeline mit Mindestanforderungen für Pull Requests. Einführung von Performance-Benchmarks für kritische Operationen wie Log-Parsing. Regelmäßige Test-Code-Reviews zur Verbesserung von Wartbarkeit und Qualität.

7.3. Fazit

Diese Arbeit analysierte das Testkonzept von lazygit und erweiterte es durch eigene Testfälle. Lazygit verfügt über eine ausgereifte Teststrategie mit Unit-Tests, Table-Driven Tests, Integrationstests und robuster CI/CD-Pipeline.

Die durchgeführten Arbeiten umfassten Identifikation von Testlücken in Tag-Kommandos und StringStack, Implementierung von 18 Testfällen mit 13 Sub-Tests für Tags und fünf für Stack-Operationen, sowie Erstellung umfassender Dokumentation. Alle Tests wurden erfolgreich in die CI-Pipeline integriert.

Die theoretischen Grundlagen des Software-Testens wurden praktisch angewendet. Table-driven Tests demonstrieren Go-Best-Practices für maximale Testabdeckung mit minimalem Code-Overhead. Lazygit dient als exzellentes Beispiel für durchdachte Teststrategien in Open-Source-Projekten. Die erstellten Tests und Dokumentationen verbessern die Codequalität und erleichtern zukünftigen Mitwirkenden den Einstieg.

Quellen

- [1] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Auflage. Spektrum Akademischer Verlag, 2009.

- [2] D. W. Hoffmann, *Software-Qualität*, 2. Auflage. Springer Vieweg, 2013.
- [3] The Go Authors, „Table Driven Tests“. Zugegriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://github.com/golang/go/wiki/TableDrivenTests>